CONSTRUCTOR UNIVERSITY

ADC-Aware Quantization of LLaMA-Family Models for In-Memory Computing

by

Alina Burykina

Master Thesis in Computer Science and Software Engineering

Submission: April 30, 2026

Supervisor: Prof. Dmitry Vetrov

Constructor University | Constructor Institute of Technology

Statutory Declaration

Family Name, Given/First Name	Burykina, Alina
Matriculation number	00000000
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

. . . .

Date, Signature

Abstract

This thesis investigates whether post-training quantization methods can be adapted to deploy LLaMA-family models in in-memory computing systems constrained by low-precision analog-to-digital converters (ADCs). Although prior work such as SmoothQuant and FlatQuant reduces activation and weight quantization error, ADC-constrained inference introduces an additional output quantization bottleneck at the matrix-vector multiplication stage. The work develops an ADC-aware evaluation pipeline for transformer linear layers, studies the limitations of standard post-training quantization in this setting, and explores modifications of FlatQuant that explicitly account for ADC behavior. The experimental part compares naive ADC quantization, baseline FlatQuant, and ADC-aware variants across different tile sizes, quantization precisions, and transform configurations. The main goal is to identify which parts of the LLaMA architecture are most sensitive to ADC constraints, whether post-training quantization alone is sufficient, and under which conditions lightweight adaptation may be required. The outcome of the thesis is intended to clarify the limits of reconstruction-only quantization methods for analog inference and to provide practical guidance for deploying large language models in IMC environments.

Contents

Abstract iii

1 Introduction 1

2 Background and Related Work 1
- 2.1 In-Memory Computing and ADC-Constrained Inference . . . 1
- 2.2 Quantization of Neural Networks . . . 1
- 2.3 Challenges of Quantizing Transformer Models . . . 2
- 2.4 Existing Methods . . . 2
- 2.5 Research Gap . . . 2

3 Problem Statement and Research Questions 2
- 3.1 Problem Definition . . . 2
- 3.2 Research Questions . . . 3
- 3.3 Scope and Assumptions . . . 3
- 3.4 Evaluation Criteria . . . 3

4 Proposed Method 3
- 4.1 Post-Training Quantisation . . . 3
- 4.2 Quantisation-Aware Adaptation via Post-ADC Residual LoRA . . . 7

5 Experiments and Results 8
- 5.1 Model, Data, and Hardware Setup . . . 8
- 5.2 Evaluation Metrics . . . 8
- 5.3 Bipolar ADC: Ablation Results . . . 9
- 5.4 Unsigned (Unipolar) ADC: A Second Hardware Setting . . . 13

6 Discussion 16
- 6.1 Interpretation of Results . . . 16
- 6.2 Limits of PTQ Under ADC Constraints . . . 16
- 6.3 Practical Implications . . . 16
- 6.4 Threats to Validity . . . 16

7 Conclusion 17

A Detailed Experimental Configurations 19

B Additional Figures and Tables 19

C Code Organization and Reproducibility Notes 19

1 Introduction

Purpose of this chapter. Introduce the broader problem of efficient inference for large language models and motivate in-memory computing as a response to the growing cost of both computation and data movement. Explain why ADC precision becomes a bottleneck in IMC systems and why this makes deployment of LLaMA-family models particularly challenging.

What to include. This chapter should explain the practical importance of the topic, define the overall thesis objective, and state why the problem is interesting from both a machine learning and hardware perspective. It should also briefly summarize the gap between conventional digital quantization methods and the specific requirements of ADC-constrained inference.

Suggested structure.

- Motivation: efficient deployment of LLMs.
- Why IMC is attractive for large matrix-vector multiplications.
- Why limited ADC precision creates a distinct quantization problem.
- Why LLaMA-family models are a difficult case.
- A short summary of the thesis contributions.
- A roadmap of the remaining chapters.

Possible contributions paragraph. A concise contribution list at the end of the introduction could state that the thesis: (i) formulates the ADC-constrained quantization problem for LLaMA-family models, (ii) implements an ADC-aware FlatQuant pipeline, (iii) evaluates post-training quantization baselines and ablations, and (iv) analyzes when reconstruction-based PTQ is sufficient and when additional adaptation is needed.

2 Background and Related Work

2.1 In-Memory Computing and ADC-Constrained Inference

Purpose. Provide the hardware context required to understand the rest of the thesis.

What to include. Describe how in-memory computing performs computation directly within memory arrays, why the analog nature of the computation requires an analog-to-digital conversion step, and why ADC precision affects both efficiency and accuracy. This is the right place to define the intuition behind the matrix-vector multiplication pipeline in IMC and to motivate why output quantization is different from standard digital quantization.

2.2 Quantization of Neural Networks

Purpose. Establish the machine learning background.

What to include. Review the main concepts of quantization: post-training quantization (PTQ), quantization-aware training (QAT), symmetric and asymmetric quantization, per-tensor and per-channel quantization, and the straight-through estimator. Keep this section focused on concepts that are directly needed later.

2.3 Challenges of Quantizing Transformer Models

Purpose. Explain why LLMs, and especially LLaMA-family models, are harder to quantize than smaller vision or language models.

What to include. Discuss activation outliers, sensitivity of the residual stream, the role of attention and MLP projections, and why preserving hidden-state geometry is important. You can also explain why some projections, such as `o_proj` and `down_proj`, are often more sensitive to quantization error.

2.4 Existing Methods

Purpose. Position the thesis relative to the state of the art.

What to include. Review methods such as SmoothQuant, FlatQuant, and ADC-oriented ideas such as activation shifting, weight reshaping, bit augmentation, and lightweight adaptation methods. For each method, answer three questions: what problem it solves, why it may help under ADC constraints, and what limitation remains for the current setting.

2.5 Research Gap

Purpose. End the chapter by identifying the precise gap that motivates your own work.

What to include. The main message here can be that existing PTQ methods primarily reduce activation and weight quantization error, whereas ADC-constrained inference introduces a separate output quantization bottleneck that is not explicitly optimized by standard reconstruction-based approaches.

3 Problem Statement and Research Questions

3.1 Problem Definition

Purpose. Formally define the problem studied in the thesis.

What to include. Introduce the linear operation in quantized code space, define the ADC-constrained output quantization step, and explain the roles of the main parameters such as activation precision, weight precision, ADC precision, tile size, and the hardware parameter k. Clearly state that the goal is to preserve model quality under these constraints.

3.2 Research Questions

Suggested research questions.

1. How strongly does limited ADC precision degrade the quality of LLaMA-family models compared with conventional quantization?
2. To what extent can a FlatQuant-style post-training approach mitigate this degradation?
3. Does reconstruction-only PTQ remain sufficient under ADC constraints, or are ADC-aware objectives required?
4. Which layers and projections are most sensitive to ADC quantization?
5. Under what conditions does the problem require lightweight model adaptation beyond PTQ?

3.3 Scope and Assumptions

Purpose. Clarify what the thesis covers and what it does not cover.

What to include. State which LLaMA-family models are evaluated, what hardware behavior is modeled, which parts of the inference pipeline are abstracted, and whether the study focuses on PTQ only or also includes limited adaptation baselines. This subsection is also the right place to state that the work relies on simulation rather than fabricated hardware.

3.4 Evaluation Criteria

Purpose. Define how success will be measured.

What to include. Include both model-level and ADC-level metrics. Model-level metrics may include perplexity or task accuracy. Internal metrics may include layerwise reconstruction error, clipping rate, dead-zone rate, bin utilization, or sensitivity by projection type.

4 Proposed Method

4.1 Post-Training Quantisation

4.1.1 Baseline: FlatQuant for LLaMA

FlatQuant [1] exploits the linear invariance of the matrix-vector product to redistribute the dynamic range of activations before quantisation. For any invertible transform $T \in \mathbb{R}^{d \times d}$,

$$y = xW^\top = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^\top)}_{\tilde{W}^\top}.$$

Choosing T to flatten the distribution of $\tilde{x}$ improves both integer quantisation accuracy and ADC bin coverage. Computing a full $d \times d$ transform is prohibitive, so T is Kronecker-factorised as $T = T_1 \otimes T_2$ with $T_1, T_2 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}$, reducing the parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$.

Calibration proceeds layer by layer in a teacher-student fashion. For each linear projection, a frozen floating-point (FP) teacher produces reference outputs $y^\star = L_{\mathrm{FP}}(x^\star)$, and the quantised student $\hat{y} = \hat{L}_T(\hat{x})$ minimises the mean-squared error $\|\hat{y} - y^\star\|^2$ over the Kronecker factors only, with weights held at their quantised values. After calibration, T is folded into the weights via reparameterisation; inference cost equals that of standard integer quantisation.

We adapt FlatQuant to the LLaMA-3 architecture [2] by wrapping all seven linear projections in each decoder block: q, k, v, o in the grouped-query attention sub-layer, and `gate`, `up`, `down` in the SwiGLU feed-forward network.

4.1.2 ADC Hardware Model and Forward Simulation

In an analog in-memory compute (AIMC) accelerator, quantised weights $\hat{w} \in [-q_w, q_w]^M$ are stored on a crossbar tile of size M. At inference, a quantised activation tile $\hat{x} \in [-q_x, q_x]^M$ is presented, the crossbar accumulates their integer dot product $y_{\mathrm{int}} = \langle \hat{w}, \hat{x} \rangle$ in the analog domain, and an ADC samples the result with a fixed step size δ:

$$\hat{y} = \mathrm{clip}\Big(\Big\lfloor \frac{y_{\mathrm{int}}}{\delta} \Big\rfloor,\ -2^{b_a-1},\ 2^{b_a-1}-1 \Big).$$

The step size is a hardware constant set at manufacturing time:

$$\delta = \frac{2\, M\, q_x\, q_w}{2^{b_a}\, k},$$

where b_a is the ADC bit-width and k is the number of ADC channels operated in parallel. Two failure modes follow directly. The *dead zone*: if $|y_{\mathrm{int}}| < \delta$, the floor operation maps the output to zero, discarding the signal entirely. *Clipping*: if $|y_{\mathrm{int}}| > \delta \cdot (2^{b_a-1}-1)$, the output saturates. For our target configuration ($b_w = b_x = 4$, $M = 256$, $b_a = 8$, $k = 16$) this gives $\delta \approx 6.12$, coarse enough that a naïve INT4 baseline shows a bypass-to-ADC perplexity gap of $\times$153 (15.41 vs. 2354).

4.1.3 Limitations of Per-Layer Reconstruction

Standard FlatQuant minimises per-projection MSE feeding each layer its *clean* FP activations from the calibration set. Two gaps make this insufficient under ADC inference. First, the calibration objective is blind to the ADC step: a projection optimised in isolation may produce pre-ADC signals that fall inside the dead zone after re-quantisation of the next tile. Second, error accumulates across layers at inference but not during calibration — the input to layer ℓ at inference is the noisy, ADC-distorted output of layer $\ell{-}1$, not the clean FP tensor the optimiser saw. These gaps motivate all three ADC-aware extensions described below.

4.1.4 Block-wise Calibration with Propagation and α-Mixing

Block-level reconstruction. Rather than minimising per-projection MSE independently, we reconstruct the output of each entire transformer block:

$$\mathcal{L} = \big\| \hat{B}(\hat{x}) - B_{\mathrm{FP}}(x^\star) \big\|_2^2.$$

Gradients propagate through the full block ($q/k/v \to \mathrm{attn} \to o \to \mathrm{gate/up} \to \mathrm{down}$), so the transforms of earlier projections are aware of how their error propagates to later ones within the same block.

Propagation across blocks. Blocks are calibrated sequentially from layer 0 to layer $N-1$. When training layer ℓ, its input is the ADC-quantised output produced by the already-trained layer $\ell-1$, not a clean FP activation. Error thus accumulates during calibration in the same way it does at inference, closing the distribution mismatch between training and deployment that per-layer calibration leaves open.

α-mixed objective. A purely ADC-propagated loss produces noisy gradients because the straight-through estimator must pass through multiple floor operations. We interpolate between the FP and ADC losses with a scalar $\alpha \in [0, 1]$:

$$\mathcal{L}(\alpha) = (1 - \alpha)\,\mathcal{L}_{\mathrm{FP}} + \alpha\,\mathcal{L}_{\mathrm{ADC}},$$

where $\mathcal{L}_{\mathrm{FP}}$ uses the clean FP input and $\mathcal{L}_{\mathrm{ADC}}$ uses the quantised input from the previous block. $\alpha = 0$ recovers clean calibration; $\alpha = 1$ is full propagation. An α-sweep (Section 5) shows that $\alpha = 0.5$ minimises ADC perplexity: below 0.5 the loss under-weights the ADC regime; above 0.5 gradient noise degrades transform quality. At $\alpha = 0.5$, INT8 ADC perplexity drops from 28.86 to 14.41 with no additional parameters.

4.1.5 Trainable Per-Channel Diagonals

We first establish a formal bound showing that orthogonal Kronecker transforms cannot suppress per-channel outliers beyond a fixed threshold, and that a diagonal factor is the precise missing degree of freedom.

Lemma 4.1 (Rotation-only outlier suppression bound)**.** *Let $P = L \otimes R$ with $L^\top L = I$, $R^\top R = I$ be an orthogonal Kronecker transform on $\mathbb{R}^d$ ($d = a \cdot b$, $L \in \mathbb{R}^{a\times a}$, $R \in \mathbb{R}^{b\times b}$). For any $x \in \mathbb{R}^d$ define the effective participation ratio*

$$m_{\mathrm{eff}}(x) \;=\; \frac{\|x\|_2^2}{\|x\|_\infty^2} \;\in\; [1,\, d].$$

Then for every orthogonal Kronecker transform P,

$$\|xP\|_\infty \;\geq\; \frac{\|x\|_2}{\sqrt{d}},$$

and consequently the maximum achievable suppression of the peak activation over all such P satisfies

$$\frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \;\leq\; \sqrt{\frac{d}{m_{\mathrm{eff}}(x)}}.$$

Proof. Since P is orthogonal, $\|xP\|_2 = \|x\|_2$. For any vector $y \in \mathbb{R}^d$ the standard ℓ^∞–ℓ^2 inequality gives $\|y\|_\infty \geq \|y\|_2/\sqrt{d}$. Applying this to $y = xP$ yields the first inequality. The suppression ratio bound then follows by dividing $\|x\|_\infty = \|x\|_2/\sqrt{m_{\mathrm{eff}}(x)}$ by the lower bound $\|xP\|_\infty \geq \|x\|_2/\sqrt{d}$. $\square$

Corollary 4.2 (Kronecker capacity)**.** *A full orthogonal matrix on $\mathbb{R}^d$ has $d(d-1)/2$ free parameters. The Kronecker family $L \otimes R$ has only $a(a-1)/2 + b(b-1)/2$ free parameters — for $d = 2048 = 32 \times 64$ this is 2512 vs. $\approx 2.1 \times 10^6$. Consequently, the Kronecker family is a strict low-dimensional subset of all orthogonal transforms and may not attain the bound in Lemma 4.1; the true achievable suppression ratio can be strictly smaller.*

Implications for INT4 ADC. In our setting, per-token activation scaling sets $s_x = \|x\|_\infty / q_{\max}$. If a small number of channels persistently dominate (i.e. $m_{\text{eff}} \ll d$), Lemma 4.1 gives a ceiling on how much rotation alone can help. For example, with $d = 2048$ and 8 dominant channels ($m_{\text{eff}} \approx 8$), the maximum suppression factor is $\sqrt{2048/8} = 16$; if the required suppression exceeds this, rotation-only calibration is provably insufficient. The INT4 baseline confirms this empirically: the ADC perplexity gap of $\times 153$ persists even after FlatQuant rotation, whereas adding diagonal scaling reduces the dead-zone rate from 80.9% to 10.4%.

Motivation and formulation. A Kronecker rotation reshapes the *covariance* of an activation cloud but does not adjust per-channel scales. Under INT4, residual outlier channels remain after rotation: a few channels with magnitudes 5–$10\times$ the median force the per-token scale factor to waste range on those channels and under-represent the rest, which directly worsens the ADC dead-zone rate because y_{int} per output channel depends on how uniformly codes are distributed.

We absorb a trainable diagonal $D = \text{diag}(d_1, \ldots, d_c)$, $d_i \in [10^{-4}, 10]$, into the rotation:

$$\tilde{T} = T \cdot D.$$

The linear map is preserved by applying equal-and-opposite rescaling to the weights:

$$y = \underbrace{(x\, D^{-1} T^{-1})}_{\tilde{x}}\, \underbrace{(T\, D\, W^\top)}_{\tilde{W}^\top}.$$

The bounds on d_i prevent degenerate scaling. D is trained jointly with T during block-level calibration and folded into $\tilde{W}$ at deployment, adding no inference overhead. Crucially, because D is not orthogonal, it breaks the ℓ^2-energy invariance assumed in Lemma 4.1: setting $d_j < 1$ directly reduces $\|xD\|_2$ in outlier channel j *before* the scale is computed, achieving suppression factors that no rotation can match.

Placement in attention and MLP blocks. We place separate diagonal matrices before the attention sub-layer (`diag_attn`, shared by $q/k/v$, plus a second instance before o) and before the FFN sub-layer (`diag_mlp`, shared by `gate`/`up`, plus a second before `down`). An ablation (Section 5) shows that both placements are necessary: `diag_mlp` drives the bypass-perplexity improvement while `diag_attn` closes the ADC gap; either alone leaves significant perplexity on the table.

4.1.6 Staged Training

Training all Kronecker factors and diagonals jointly from scratch with $\alpha = 0.5$ is unstable: attention gradients dominate early optimisation before MLP transforms have converged, yielding a poor initialisation for both sub-systems.

We split calibration into two stages. *Stage A* (30 epochs, learning rate η) trains only the MLP Kronecker factors $T_{\text{gate}}, T_{\text{up}}, T_{\text{down}}$ and MLP diagonals $D^{(\text{m})}$ with $\alpha = 0$ (clean FP inputs), providing a stable, noise-free landscape in which FFN transforms converge. *Stage B* (10 epochs, learning rate 0.1η) unfreezes the attention Kronecker factors T_q, T_k, T_v, T_o and attention diagonals $D^{(\text{a})}$, and switches to $\alpha = 0.5$. The MLP warm start prevents gradient interference from attention. This procedure achieves the best INT4 results among all pure-PTQ configurations: bypass perplexity 18.50, ADC perplexity 27.60.

4.2 Quantisation-Aware Adaptation via Post-ADC Residual LoRA

PTQ methods reach a floor around 27.5–28.5 ADC perplexity for INT4 regardless of calibration strategy. The limiting factor is ADC resolution: with $\delta \approx 6.12$ each output channel has at most ~30 usable discrete levels, introducing irreducible rounding error that no linear transform can eliminate. We complement PTQ with a lightweight adaptation step that learns to correct ADC distortion *after* it has occurred, in the digital domain, without modifying the frozen quantised model.

4.2.1 Architecture

The PTQ checkpoint is frozen entirely. For each of the seven linear projections in every decoder block, we add a parallel low-rank branch that reads the full-precision input x_{fp} and adds its output to the ADC result:

$$y = \underbrace{\mathrm{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_r}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r} \quad ,$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are LoRA [3] factors stored and computed in `float32`, and α_r / r is the standard LoRA scaling. All base model parameters — weights, Kronecker factors, diagonals, and ADC step sizes — remain frozen throughout. For Llama-3.2-1B ($d = 2048$, 16 decoder blocks, 7 projections each), rank-4 adapters add ~1.8M parameters (0.15% of the total model).

4.2.2 Training Objective

The adapters are trained for 5 epochs with learning rate 10^{-4} on the same 128-sample calibration set used for PTQ. The loss combines cross-entropy on the model's predictions with a knowledge-distillation term towards the FP teacher:

$$\mathcal{L}_{\text{LoRA}} = \mathrm{CE}(\hat{p},\, y) + \lambda \, \mathrm{KL}\big(\hat{p} \,\big\|\, p_{\text{FP}}\big)\,,$$

with $\lambda = 1$. The FP teacher forward pass is computed once per batch with no gradient. Ablations (Section 5) show that the KL term improves ADC perplexity by ~0.6 points over CE alone, and that rank $r = 4$ saturates the gain — rank 8 yields only marginal further improvement. Restricting adapters to the last eight layers is significantly worse than covering all sixteen, confirming that ADC distortion accumulates across the full depth of the network.

4.2.3 Why Post-ADC Placement

Placing the LoRA correction *before* the ADC — modifying the pre-ADC integer signal — fails catastrophically. Any correction $\Delta = BAx$ is itself quantised by the fixed step δ before reaching the output, so it cannot express sub-δ adjustments. Moreover, gradients through the floor operator are too noisy for stable training from a cold start, and the coupled interaction with the hard ADC clamp causes divergence.

Post-ADC placement avoids both problems: the correction is added in the digital domain after the ADC read, at full floating-point precision, with clean gradients from the CE+KL loss. ADC perplexity drops from the PTQ plateau of 27.60 to 13.96 for INT4 on WikiText-2, surpassing the INT8 PTQ result (14.41), and generalises to C4 (ADC perplexity 23.30 vs. PTQ baseline 46.40).

5 Experiments and Results

5.1 Model, Data, and Hardware Setup

Model. All experiments use Llama-3.2-1B [2] in `bfloat16`. The model has 16 decoder layers, hidden dimension $d = 2048$, 32 attention heads with grouped-query attention, and a SwiGLU FFN with intermediate dimension 8192. It is small enough to run a full sweep of dozens of configurations overnight on a single GPU while being representative of the transformer architecture used in larger LLaMA-family models.

Calibration data. FlatQuant transforms and LoRA adapters are calibrated on WikiText-2 training text [4]. Unless stated otherwise, 128 randomly sampled sequences of 2048 tokens each are used. Selected experiments use 512 or 1024 samples to study the effect of calibration set size; the INT4 series demonstrates that sample count is a dominant lever for this bit-width.

Evaluation. Perplexity is measured on the WikiText-2 test set using a sliding window of context length 2048 and stride 1024. Selected configurations are additionally evaluated on C4 [5] under the same protocol to verify out-of-domain generalisation.

ADC hardware parameters. Table 1 lists the fixed hardware constants used throughout. All values are chosen to match a plausible near-term AIMC chip target. The ADC step size δ is fully determined by these constants via $\delta = 2Mq_xq_w/(2^{b_a}k)$, giving $\delta \approx 2016$ for INT8 and $\delta \approx 6.12$ for INT4.

Table 1: Fixed ADC hardware parameters.

Parameter	**Symbol**	**Value**
Tile size (crossbar columns)	M	256
ADC bit-width	b_a	8
Parallel ADC channels	k	16
INT8 weight range	q_w	127
INT8 activation range	q_x	127
INT4 weight range	q_w	7
INT4 activation range	q_x	7
ADC step (INT8)	δ	≈ 2016
ADC step (INT4)	δ	≈ 6.12

FlatQuant calibration settings. Transforms are trained for 30 epochs with learning rate 5×10^{-3} using the AdamW optimiser. Stage B of staged training uses 10 epochs at 5×10^{-4}. Static activation scales for Step 2 of the FlatQuant pipeline use the 99.9th-percentile clipping strategy (`calibration_method=percentile`).

5.2 Evaluation Metrics

All experiments are characterised by the following metrics.

- **PPL (bypass).** WikiText-2 perplexity with ADC disabled; only tiling and integer quantisation are active. Measures FlatQuant transform quality in isolation. Lower is better; the FP baseline is ≈ 10.5 (INT8 regime) and ≈ 10.5 / ≈ 14 depending on bit-width.
- **PPL (ADC).** WikiText-2 perplexity with the full ADC pipeline enabled. This is the primary metric; the bypass–ADC gap measures the additional degradation introduced by finite-resolution ADC.
- **dead_rate.** Fraction of output channels across all layers and calibration tokens for which $|y_{\text{int}}| < \delta$, i.e. the ADC output is identically zero. Measured on the calibration set; lower is better.
- **clip_rate.** Fraction of ADC outputs that saturate at the maximum code $\pm(2^{b_a-1}-1)$. High clip rate indicates the integer accumulation systematically overflows the ADC range.
- **bin_usage.** Fraction of the 2^{b_a} ADC output bins that are actually occupied, averaged over layers and tokens. Higher is better; near-zero bin usage indicates the ADC is effectively acting as a 1-bit comparator.
- **reconstruction_rel.** Relative mean-squared error between the ADC output and the FP reference, $\|\hat{y}-y^\star\|^2/\|y^\star\|^2$, on the calibration set. Tracks layer-level signal fidelity independently of perplexity.

5.3 Bipolar ADC: Ablation Results

All results below use the bipolar ADC with signed accumulation $y_{\text{int}} \in [-M, +M]$ and the hardware constants from Table 1. FP reference: WikiText-2 PPL 10.5. All tables report WikiText-2 perplexity unless stated otherwise.

5.3.1 INT8: Effect of Calibration Objective

Table 2 compares four INT8 configurations at 128 calibration samples, isolating the contribution of block-wise propagation and the bin-center auxiliary loss.

Table 2: INT8 calibration study. Llama-3.2-1B, 128 samples, $\delta \approx 2016$.

Configuration	Bypass PPL	ADC PPL	dead_rate
FP baseline	—	10.5	—
Baseline ($\alpha = 0$)	10.01	28.86	10.3%
Bin-center ($\lambda = 0.1$)	10.01	28.86	10.3%
Propagated ($\alpha = 1$)	11.01	**14.55**	10.3%
Prop + bin-center	11.00	**14.41**	10.3%

Propagation alone reduces ADC PPL by 47% (28.86 $\rightarrow$ 14.55) with no change to dead_rate, confirming that the remaining gap is ADC resolution, not the dead zone. The bin-center loss has zero effect on its own but gives a marginal improvement (+0.14 PPL) on top of propagation.

5.3.2 INT4: Sample Count and Propagation

The INT4 baseline exhibits a bypass–ADC gap of $\times$153 (15.41 vs. 2354.86), far wider than INT8. Table 3 shows that 128 calibration samples are insufficient for propagation, while 512 samples combined with propagation collapse the gap to $\times$1.2.

Table 3: INT4 sample count and propagation sweep. $\delta \approx 6.12$.

Config	Samples	Bypass PPL	ADC PPL	Gap
Baseline	128	15.41	2354.86	$\times$153
Propagated ($\alpha = 1$)	128	40.28	203.89	$\times$5.1
Hadamard init + prop	128	34.73	178.86	$\times$5.1
Decoupled (train w8, eval w4)	128	11.64	276.25	$\times$24
Propagated ($\alpha = 1$)	512	34.25	**40.63**	$\times$1.2
Baseline	1024	19.80	56.83	$\times$2.9

Hadamard initialisation provides no benefit over random initialisation at the same sample count. Decoupled training (transforms optimised for INT8, evaluated at INT4 ADC) achieves the best bypass PPL (11.64) but a poor ADC gap ($\times$24) because the transforms are not calibrated to the INT4 δ.

5.3.3 INT4: α-Mixing Sweep (1024 Samples)

Table 4 sweeps α from 0 to 1 at 1024 samples and evaluates two additional configurations: two-stage training and the diagonal extension.

Table 4: INT4 α-mixing sweep. 1024 calibration samples.

Config	α	Bypass PPL	ADC PPL	Gap
Baseline	0.00	19.80	56.83	$\times$2.87
Prop $\alpha = 0.25$	0.25	21.65	32.35	$\times$1.49
Prop $\alpha = 0.5$	0.50	24.24	31.22	$\times$1.29
Prop $\alpha = 0.75$	0.75	27.46	35.92	$\times$1.31
Prop $\alpha = 1.0$ (full)	1.00	30.32	40.20	$\times$1.32
2-stage (A: no prop, B: $\alpha = 0.5$)	—	22.82	32.21	$\times$1.41
Diag + $\alpha = 0.5$	0.50	**20.23**	**27.56**	$\times$1.36

$\alpha = 0.5$ gives the best gap ratio ($\times$1.29); below 0.5 the ADC regime is under-weighted, above 0.5 gradient noise from the floor operator degrades transform quality. Adding the diagonal (bounded LET, $d_i \in [10^{-4}, 10]$) with $\alpha = 0.5$ achieves the best absolute ADC PPL (27.56) across all pure-PTQ configurations.

5.3.4 INT4: Diagonal Placement, MLP Decomposition, and Staged Training

Table 5 ablates diagonal placement (v3) and decomposes the MLP diagonal into its two components (v4), identifying the `down_proj` as the key driver.

Table 5: INT4 diagonal placement and staged training. 1024 samples, $\alpha = 0.5$.

Config	Bypass PPL	ADC PPL	Note
Both diag (control)	19.69	28.46	reproduced v2
Attn diag only	24.98	29.41	worse bypass
MLP diag only	19.37	31.65	best bypass, wide gap
MLP diag + layer-wise α	**19.00**	28.66	best bypass overall
Both diag + layer-wise α	19.39	28.55	no gain over flat α
`up_gate` diag only	23.22	39.59	weak
`down` diag only	20.42	31.26	key driver
Staged: MLP diag $\rightarrow$ attn diag	**18.50**	**27.60**	**best PTQ**

The `down_proj` diagonal drives nearly all of the MLP bypass improvement; `up/gate` contributes little. Staged training (Stage A: MLP diag, no prop; Stage B: attn diag, $\alpha = 0.5$) achieves the best combined result — bypass 18.50 and ADC 27.60 — by avoiding gradient interference between the two sub-systems.

5.3.5 INT4: Stochastic Propagation

Table 6 compares deterministic $\alpha = 0.5$ against two stochastic mixing schedules, in flat and staged settings.

Table 6: INT4 stochastic propagation. 1024 samples.

Schedule	Mode	Bypass PPL	ADC PPL	Gap
Deterministic $\alpha = 0.5$	flat	24.24	31.22	$\times$1.29
Bernoulli (p=0.5)	flat	19.25	35.38	$\times$1.84
Beta(2,2)	flat	22.90	31.40	$\times$1.37
Deterministic $\alpha = 0.5$	staged	18.50	27.60	$\times$1.49
Bernoulli (p=0.5)	staged	**16.94**	32.44	$\times$1.92
Beta(2,2)	staged	17.16	**27.82**	$\times$1.62

Bernoulli switching yields the best bypass (16.94) but the worst ADC gap — batches that use only the clean FP path provide no ADC signal to the attention transforms. Beta(2,2) samples near 0.5 each batch and closely matches deterministic $\alpha = 0.5$; the marginal ADC gain (27.82 vs. 27.60) does not justify the added complexity.

5.3.6 Post-ADC Residual LoRA: Initial Sweep

Table 7 evaluates LoRA adapters on top of the best PTQ checkpoint (`staged`, ADC PPL 27.60), varying rank and target projection sets.

Across all LoRA configurations ADC PPL falls below bypass PPL (gap < 1), confirming that the adapter learns to specifically correct ADC noise. Adding `o_proj` to `down_proj`

Table 7: LoRA initial sweep (v6). Base PTQ: bypass 18.50, ADC 27.60.

Config	**Targets**	**Bypass PPL**	**ADC PPL**	**Gap**
No LoRA (base PTQ)	—	18.50	27.60	$\times$1.49
LoRA $r = 4$, `down`	1 proj	19.33	16.32	$\times$0.84
LoRA $r = 8$, `down`	1 proj	19.17	15.79	$\times$0.82
LoRA $r = 4$, `down+o`	2 proj	18.68	**15.63**	$\times$0.84
LoRA $r = 4$, all 7 proj	7 proj	**17.73**	16.53	$\times$0.93

gives the best ADC PPL (15.63); covering all 7 projections gives the best bypass (17.73) at the cost of a slightly wider gap.

5.3.7 Post-ADC Residual LoRA: Ablation Study

Table 8 reports the full ablation across rank, loss, layer coverage, and adapter placement (v7, all runs use $r = 4$, `down+o` unless stated).

Table 8: LoRA ablation study (v7). Llama-3.2-1B, INT4.

Axis	**Setting**	**ADC PPL**
Seed	seed 1	15.79
	seed 2	15.88
	seed 3	15.93
Rank	$r = 1$	15.90
	$r = 2$	15.60
	$r = 4$	**15.57**
	$r = 8$	15.24
Loss	CE only	14.60
	CE + KL	**13.96**
Layer coverage	first 8 layers	16.64
	all 16 layers	**15.63**
	last 8 layers	20.08
Placement	post-ADC (proposed)	**15.63**
	pre-ADC	105.63 *(diverged)*
All 7 proj	CE only	16.68
	CE + KL	**14.03**

Rank saturates at $r = 4$ (only 0.33 PPL gap to $r = 8$). CE+KL outperforms CE alone by 0.64 PPL with `down+o` targets and by 2.65 PPL with all-7 targets — the FP teacher signal is most valuable when the adapter covers more projections. Earlier layers carry more accumulated ADC error: first-8 (16.64) beats last-8 (20.08). Pre-ADC placement diverges catastrophically (ADC PPL 105.6) because the correction is itself quantised by the fixed δ. The best single configuration is **all-7, CE+KL,** $r = 4$: ADC PPL 14.03, beating the INT8 PTQ result (14.41).

5.3.8 Post-ADC Residual LoRA: Generalisation to C4

Table 9 evaluates the top-3 configurations on both WikiText-2 and C4 to test out-of-domain transfer.

Table 9: LoRA generalisation (v8). WikiText-2 and C4 perplexity.

Config	Wiki bypass	Wiki ADC	C4 bypass	C4 ADC
PTQ (no LoRA)	18.49	27.26	29.41	46.40
$r = 8$, `down+o`, CE	18.51	15.47	30.48	29.39
$r = 4$, `down+o`, CE+KL	19.05	14.27	29.37	23.88
$r = 4$, all 7, CE+KL	**18.70**	**13.96**	**29.10**	**23.30**

CE+KL configurations reduce C4 ADC PPL by 50% relative to the PTQ baseline (46.40 $\rightarrow$ 23.30) despite being calibrated solely on WikiText-2. CE-only ($r = 8$, `down+o`) shows much weaker C4 generalisation (29.39 vs. 23.30), confirming that the KL teacher signal is the key ingredient for cross-domain transfer. All-7 CE+KL is the best overall checkpoint on both datasets.

5.4 Unsigned (Unipolar) ADC: A Second Hardware Setting

All experiments described so far assume a *bipolar* ADC: the analog crossbar accumulates a signed integer dot product $y_{\text{int}} \in [-M, +M]$ and the ADC reads both positive and negative values. A physically important alternative is a *unipolar* (unsigned) ADC, which accepts only non-negative inputs. Real crossbar hardware is often constrained to non-negative currents, making this the more realistic operating regime. The two settings have the same bit-width $b_a = 8$ and tile size $M = 256$, but they differ in the ADC range and in how the signed dot product is recovered — which changes δ by more than a factor of two and requires a different calibration objective. We therefore treat the unipolar setting as a self-contained experiment series with its own baselines, calibration, and ablation structure.

5.4.1 Unsigned Shift-Subtract ADC Algorithm

Because the crossbar accepts only $[0, 2^b - 1]$ codes, the standard signed INT4 codes $\hat{x} \in [-8, 7]$ and $\hat{w} \in [-8, 7]$ must be shifted to unsigned before the MVM. The algorithm proceeds in five steps.

Step 1 — Shift to unsigned. Each signed code is offset by the zero-point $z = 2^{b_x - 1} = 8$:

$$\hat{x}^u = \hat{x} + 8 \ \in [0, 15], \qquad \hat{w}^u = \hat{w} + 8 \ \in [0, 15].$$

Step 2 — Hardware MVM. The crossbar computes the non-negative dot product over one tile of width $M = 256$:

$$y^u = \langle \hat{x}^u, \ \hat{w}^u \rangle \ \in \ [0, \ M(2^{b_x} - 1)(2^{b_w} - 1)] = [0, \ 57600].$$

Because $y^u \geq 0$ always, a single unsigned MVM suffices with no four-quadrant splitting.

Step 3 — ADC floor. The unipolar ADC step size is:

$$\delta_u = \frac{M\,(2^{b_x}-1)\,(2^{b_w}-1)}{2^{b_a}\,k} = \frac{256\cdot 15\cdot 15}{256\cdot 16} = 14.0625 \quad (k=16),$$

and the quantised output is $\hat{y}^u = \lfloor y^u/\delta_u \rfloor \cdot \delta_u$. Note that $\delta_u \approx 14.06$ is more than twice the bipolar $\delta \approx 6.12$ at the same k, reflecting the fact that the same 2^{b_a} bins now cover a range that is roughly $2\times$ wider in absolute terms.

Step 4 — Digital correction (exact). Expanding the unsigned product gives $\hat{x}^u \hat{w}^u = \hat{x}\hat{w} + 8\hat{w} + 8\hat{x} + 64$, so the signed inner product is recovered exactly by:

$$y_{\text{int}} = \hat{y}^u \;-\; 8\sum_j \hat{w}_j^u \;-\; 8\sum_i \hat{x}_i^u \;+\; M\cdot 64,$$

where $\sum_j \hat{w}_j^u$ is precomputed once at weight-load time and $\sum_i \hat{x}_i^u$ is computed once per token. The correction involves no approximation and adds negligible digital overhead.

Step 5 — Dequantisation. $y_{\text{real}} = y_{\text{int}} \cdot s_x \cdot s_w$, where s_x is the per-token activation scale and s_w the per-channel weight scale, exactly as in the bipolar setting.

5.4.2 Per-Layer k Search

Because the coarser δ_u (at global $k = 16$) imposes a heavier resolution penalty than the bipolar setting, we introduce a *per-layer k search* that selects the ADC parallelism independently for each projection layer.

After FlatQuant calibration with a global k, the pre-ADC values y^u are captured on the calibration set for each layer. The minimum k that keeps the ADC range from saturating is found by:

$$R_\ell = \text{quantile}_{0.999}(|y_\ell^u|), \qquad k_\ell^* = \min\left\{ k \in \mathcal{K} : k \geq \frac{M\,(2^{b_x}-1)(2^{b_w}-1)}{2\,R_\ell} \right\},$$

where $\mathcal{K} = \{4, 8, 16, 32, 64\}$. Layers with a narrow output distribution receive a small k (coarser δ, fewer ADC reads); layers with wide distributions receive a large k (finer δ, more reads). The search is run in-place on the fixed PTQ checkpoint without retraining FlatQuant transforms.

5.4.3 Calibration Objective for Unipolar ADC

FlatQuant transforms calibrated for the bipolar $\delta \approx 6.12$ are mismatched to the unipolar $\delta_u \approx 14.06$: the transforms shape the pre-ADC signal for a finer resolution that the hardware does not provide. In the unipolar series, FlatQuant is therefore *retrained* with the unipolar δ_u formula as the active ADC step during calibration. All other settings (staged training, $\alpha = 0.5$, diagonal scaling) are carried over from the best bipolar configuration.

5.4.4 Experiment Configurations

The following six configurations are evaluated for Llama-3.2-1B on WikiText-2 and C4:

- `fp` — full-precision FP16 baseline.
- `int4_no_adc` — INT4 FlatQuant with the ADC floor disabled; measures FlatQuant transform quality alone. Trained with the bipolar formula (calibration does not see ADC noise), giving WikiText-2 PPL 12.56.
- `best_ptq` — INT4 FlatQuant retrained with δ_u, global $k = 16$.
- `best_ptq_k` — `best_ptq` with per-layer k search applied in-place (no FlatQuant retraining).
- `best_ptq_k_recal` — `best_ptq_k` followed by retraining FlatQuant with the found per-layer k values baked in; tests whether retrained transforms improve over in-place k search.
- `best_lora` — `best_ptq` plus post-ADC residual LoRA ($r = 4$, all 7 projections, CE+KL loss, $\lambda = 0.5$, temperature 2.0, 5 epochs at 10^{-4}).
- `best_lora_k` — `best_lora` plus per-layer k search; the LoRA adapters are trained on top of the k-searched PTQ checkpoint.

5.4.5 Experiment History and Hardware Setting Evolution

The unipolar series was reached through three prior hardware-setting iterations, each of which exposed a design flaw in the preceding one. We document them here because they clarify why the final configuration is necessary, not arbitrary.

Bipolar ADC (signed accumulation). The original setting uses a signed ADC that reads $y_{\text{int}} \in [-M, +M]$ directly. FlatQuant calibration uses $\delta \approx 6.12$ (bipolar formula). This gives `best_ptq` WikiText-2 PPL 26.78 and `best_lora` PPL 14.01 — the reference for the signed regime.

Four-quadrant unipolar. An earlier attempt at unipolar hardware splits signed codes into positive and negative parts and runs four separate non-negative MVMs (x^+w^+, x^-w^-, x^+w^-, x^-w^+). This achieves $\delta_{\text{branch}} \approx 3.06$ (twice as fine as bipolar) at the cost of 4× the ADC reads. FlatQuant was trained for the bipolar δ, so the calibration objective was still mismatched; `best_ptq` PPL was 18.08.

Mismatched unsigned evaluation (historical). An intermediate step evaluated the unsigned shift-subtract hardware model using FlatQuant checkpoints trained for the bipolar $\delta \approx 6.12$. The mismatch caused `best_ptq` PPL to explode to 94274 because transforms shaped the pre-ADC signal for a step size six times finer than the actual hardware provides. Per-layer k search partially recovered (PPL 16.78) by selecting small k to bring δ back into range, but this was an incorrect setup and the results were not used.

Unipolar FQ — current. The correct configuration retrains FlatQuant with the unipolar δ_u formula from the start. Table 10 summarises the results. `best_ptq_k_recal` is *worse* than `best_ptq_k`, indicating that retraining FlatQuant on fixed per-layer k values over-specialises the transforms and hurts generalisation; in-place k search after global-k training is preferable.

Table 10: Unipolar (unsigned) ADC results on Llama-3.2-1B. $\delta_u = 14.0625$ at global $k = 16$.

Config	Description	Wiki PPL	C4 PPL
`fp`	FP16, no quantisation	**8.68**	**13.13**
`int4_no_adc`	INT4 FQ, ADC disabled	12.56	20.13
`best_ptq`	INT4 FQ + unipolar ADC, k=16	39.00	67.78
`best_ptq_k`	+ per-layer k search	21.64	33.79
`best_ptq_k_recal`	+ FQ retrained with per-layer k	22.19	36.94
`best_lora`	+ post-ADC LoRA (global k=16)	17.29	29.26
`best_lora_k`	+ LoRA & per-layer k	**14.56**	**23.78**

6 Discussion

6.1 Interpretation of Results

Purpose. Move from raw results to explanation.

What to include. Discuss why certain modifications work and others do not. Relate the observed behavior back to the theory of ADC-constrained output quantization and the internal statistics of LLaMA layers.

6.2 Limits of PTQ Under ADC Constraints

Purpose. State clearly where the chosen approach succeeds and where it fails.

What to include. This subsection should answer whether reconstruction-based PTQ remains sufficient or whether the problem fundamentally requires some degree of model adaptation in the low-bit regime.

6.3 Practical Implications

Purpose. Explain what the results mean beyond the specific experiments.

What to include. Discuss what your findings imply for IMC deployment, for the design of ADC-aware quantization methods, and for future hardware-software co-design of transformer inference systems.

6.4 Threats to Validity

Purpose. Strengthen the scientific quality of the thesis.

What to include. Mention limitations such as reliance on simulation, restricted model sizes, calibration dataset choice, and the gap between simplified hardware assumptions and a real chip implementation.

7 Conclusion

Purpose. Provide a concise answer to the research questions and summarize the thesis contributions.

What to include. Restate the original problem, summarize the method and experimental findings, state the main conclusion about ADC-aware PTQ for LLaMA-family models, and list a few concrete directions for future work such as better output-aware objectives, mixed precision, or lightweight adaptation.

References

[1] Yuxuan Chen et al. "FlatQuant: Flatness Matters for LLM Quantization". In: *arXiv preprint arXiv:2410.09426* (2024). URL: https://arxiv.org/abs/2410.09426.

[2] Abhimanyu Dubey et al. "The Llama 3 Herd of Models". In: *arXiv preprint arXiv:2407.21783* (2024). URL: https://arxiv.org/abs/2407.21783.

[3] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022. URL: https://arxiv.org/abs/2106.09685.

[4] Stephen Merity et al. "Pointer Sentinel Mixture Models". In: *International Conference on Learning Representations (ICLR)*. 2017. URL: https://arxiv.org/abs/1609.07843.

[5] Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: https://arxiv.org/abs/1910.10683.

A Detailed Experimental Configurations

Suggested contents. Put complete hyperparameter tables here: number of calibration samples, batch size, learning rates, transform settings, clipping settings, hardware parameters, and final selected configurations for each run.

B Additional Figures and Tables

Suggested contents. Add extended ablation tables, per-layer plots, extra sensitivity analyses, and any visualizations that support the main text but are too large to place in the core chapters.

C Code Organization and Reproducibility Notes

Suggested contents. Briefly describe the repository structure, entry points for training and evaluation, external dependencies, and the exact steps required to reproduce the reported results.